Excelsis 360

Attendance Analyst Agent

Technical Documentation · v1.0

Technical Documentation

An AI-powered attendance analysis platform for schools,
built on locally-hosted LLMs, SQL Server, and a React web interface.

Version 1.0 · May 2026

Table of Contents

1. Project Overview

The Excelsis 360 Analyst Agent is an AI-powered attendance analysis tool built for the Excelsis 360 platform. It extends the platform's existing data infrastructure with a reasoning AI layer — combining a locally-hosted large language model (LLM), direct access to Excelsis 360's SQL Server attendance data, and a dedicated web interface — to give school staff a conversational way to interrogate attendance records without writing reports or navigating complex dashboards.

Where the Excelsis 360 platform manages attendance records and surfaces pre-built reports, the Analyst Agent sits on top of that data and answers questions in plain English. A teacher can ask "which students in 10A are at risk this month?" and the agent reasons across the data, selects the right query strategy, and streams back a specific, cited answer — without any hard-coded report logic and without the user needing to know SQL or navigate pivot tables.

1.1 Core Principles

- **Privacy-first.** All inference runs locally via Ollama — no attendance data ever leaves the school network.
- **Read-only data access.** The SQL layer enforces SELECT-only queries through both AST-level SQL parsing and allow-listed databases. The AI agent cannot modify, insert, or delete any records.
- **Transparent reasoning.** The streaming interface shows each tool call as it happens, letting staff see exactly which data sources the agent consulted before producing an answer.
- **Zero speculative statistics.** The system prompt instructs the model never to fabricate data — if the data is not in the store, the agent says so explicitly.
- **Minimal footprint.** No cloud dependencies at inference time. The only external service is Ollama running on localhost.

1.2 Key Features

Feature	Description
`Natural-language chat`	Ask questions in plain English; the agent reasons and streams the answer token-by-token
`ReAct reasoning loop`	The model decides which tools to call, in what order, across multiple steps
`At-risk detection`	Automatic flagging of students below a configurable attendance threshold (default 75%)
`Ad-hoc SQL`	The agent can write and run T-SQL SELECT queries against any configured database
`Live dashboard`	Interactive charts: class attendance, weekly trend, status donut, day-of-week bar, trend comparison
`Agent ↔ dashboard link`	Asking the agent to show a chart updates the dashboard in real time (SSE event)

Feature	Description
`User management`	Admin-controlled user accounts with bcrypt-hashed passwords and JWT tokens
`MCP server`	Exposes attendance tools directly to Claude Code via the Model Context Protocol
`Jupyter notebook`	Full interactive analysis environment sharing the same Python backend
`Rate limiting`	Chat endpoint limited to 10 requests per minute per IP via SlowAPI

2. Use Cases

Excelsis 360 is built around several concrete workflows that school staff encounter daily. Each use case maps to one or more agent tools and front-end views.

2.1 Daily Attendance Check

A homeroom teacher wants a quick overview of which classes underperformed today or this week.

- Step 1: Open the Chat page and type: "Which classes had the lowest attendance this week?"
- Step 2: The agent calls query_attendance with period=last_7_days, group_by=class.
- Step 3: Results are streamed back as a ranked table of classes with attendance rates.
- Step 4: The agent flags any class below 75% as at-risk.

2.2 At-Risk Student Identification

A school counsellor needs to identify students who are chronically absent and may need intervention.

- Step 1: Ask: "List all students below 70% attendance in class 10A."
- Step 2: The agent calls get_at_risk_students with threshold=70.0.
- Step 3: A table is returned with student IDs, names, class, total sessions, and attendance rate.
- Step 4: The Dashboard's at-risk panel also shows this data with sparkline trend charts per student.

2.3 Trend Analysis

A deputy principal wants to know if attendance has improved or declined over the past month compared to the prior month.

- Step 1: Ask: "How does attendance this month compare to the previous month?"
- Step 2: The agent calls compare_periods(period_a='last_30_days', period_b='prior_30_days').
- Step 3: A side-by-side table with a delta column is returned, highlighting classes that improved or declined.
- Step 4: The TrendComparisonChart on the Dashboard visualises this automatically.

2.4 Class Comparison

A head of year wants to benchmark two specific classes against each other.

- Step 1: Ask: "Compare 10A and 10B attendance."
- Step 2: The agent calls compare_classes(class_a='10A', class_b='10B').
- Step 3: Both classes are presented side by side with present, absent, late counts and overall rate.

2.5 Ad-hoc Reporting

An administrator needs a custom report — for example, absences on Monday across all grades.

- Step 1: Ask: "Which day of the week has the most absences?"
- Step 2: The agent calls query_attendance with group_by=day_of_week.
- Step 3: For more complex needs, the agent writes a T-SQL query via run_sql_query.
- Step 4: Example: SELECT DATENAME(dw,date) AS day, COUNT(*) AS absences FROM attendance WHERE status='absent' GROUP BY DATENAME(dw,date) ORDER BY absences DESC

2.6 Dashboard-Driven Exploration

A staff member wants to explore data visually rather than through chat.

- Step 1: Navigate to the Dashboard page.
- Step 2: Use the FilterBar to select specific classes and time periods.
- Step 3: Click a bar in the Attendance by Class chart to highlight that class across all charts.
- Step 4: Double-click to drill down to student-level data for that class.
- Step 5: Click a student row to inspect their weekly trend sparkline.
- Step 6: Use 'Ask Excelsis' to open the embedded chat panel and ask follow-up questions — the agent can update the dashboard view directly.

2.7 Policy Knowledge

A teacher asks about best-practice intervention strategies for chronic absenteeism.

- Step 1: The agent answers conversationally from its training knowledge without calling any tools.
- Step 2: Responses include evidence-based approaches such as early warning systems, family engagement, and attendance contracts.
- Step 3: This requires no data and demonstrates the dual role of the model: data analyst and domain expert.

2.8 Claude Code Integration (MCP)

A developer or power user wants to query attendance data directly from Claude Code.

- Step 1: Start the MCP server: python -m src.mcp_server
- Step 2: Configure Claude Code to connect to the stdio server.
- Step 3: Claude Code can now call ask_analyst, attendance_summary, at_risk_students, and class_statistics as tools.
- Step 4: This enables attendance analysis within AI-assisted development workflows.

3. System Architecture

3.1 Request Flow

Every user interaction follows the same path through the system. Understanding this flow is key to understanding how all the components fit together.

Layer	Component	Technology	Role
`React App`	React 18 + Vite	Renders UI, manages auth token, opens SSE stream	
`Fetch / Axios`	Browser Fetch API	POST /chat/stream → EventSource-style reader	
`FastAPI`	Python / Uvicorn	JWT validation, rate limiting, request routing	
`ExcelsisAgent`	LangGraph ReAct	Maintains conversation history, streams LLM events	
`phi4:14b`	Ollama / ChatOllama	Reasons about which tools to call and in what order	
`7 LangGraph tools`	Python functions	Execute SQL queries and compute statistics	
`SQLAttendanceStore`	pyodbc / SQL Server	Read-only T-SQL queries with TTL cache	
`SQL Server`	ODBC Driver 18	Primary attendance table + optional extra databases	

3.2 Component Diagram (textual)

```
Browser (React :5173)
  ■
  ■  POST /chat/stream  (Bearer JWT)
  ■  GET  /data/summary, /data/at-risk, /data/stats …
  ▼
FastAPI (:8000)
  ■■■ /auth/*        JWT sign/verify, user CRUD
  ■■■ /chat/stream   SSE — delegates to ExcelsisAgent.astream_events()
  ■■■ /data/*        Direct SQLAttendanceStore queries (no LLM)
```

```
ExcelsisAgent  (LangGraph ReAct)
  ■■■ ChatOllama ■■■ phi4:14b @ http://localhost:11434
  ■■■ Conversation history (last 10 turns)
  ■■■ 7 tools ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                                                                    ■
SQLAttendanceStore (read-only)                         Tools        ■
  ■■■ _assert_select_only()  ← sqlglot AST guard        ■■■■■■■■■■■■■■■
  ■■■ _TTLCache (5-min TTL)
  ■■■ pyodbc ■■■ SQL Server
        ■■■ primary_db  (attendance table)
        ■■■ extra_db_1, extra_db_2, …  (allowlisted)

MCP Server (stdio)
  ■■■ FastMCP ■■■ ExcelsisAgent + SQLAttendanceStore
        ■■■ ask_analyst()
        ■■■ attendance_summary()
        ■■■ at_risk_students()
        ■■■ class_statistics()
```

3.3 Streaming Architecture

The chat endpoint uses Server-Sent Events (SSE) to stream the agent's output to the browser incrementally. The event types emitted are:

SSE Event	Frontend Action
`{"type": "token", "content": "…"}`	One LLM output token — appended to the current message bubble
`{"type": "tool_start", "tool": "…"}`	An agent tool call has started — UI shows a spinner badge
`{"type": "tool_end", "tool": "…"}`	The tool call returned — spinner removed
`{"type": "dashboard_filter", …}`	Agent called update_dashboard_view — Dashboard state updates live
`{"type": "error", "message": "…"}`	Timeout or model error — displayed as an error message
`{"type": "done"}`	Stream complete — UI unlocks the input field

4. Technology Stack

Every library in the stack was chosen for a specific reason. The guiding principle was to keep the dependency surface small while covering all required functionality.

Library	Package	Layer	Why chosen
`phi4:14b`	Ollama	LLM	14B parameter model; strong tool-calling and instruction-following; fits on a single consumer GPU or Apple Silicon Mac
`LangGraph`	langgraph	Agent framework	ReAct loop with built-in conversation state, tool routing, and async event streaming
`langchain-ollama`	langchain-ollama	LLM bridge	ChatOllama wraps Ollama's HTTP API with LangChain's standard interface
`FastAPI`	fastapi	Web framework	Async, schema-validated, auto-docs; minimal boilerplate for REST + SSE
`pyodbc`	pyodbc	SQL driver	ODBC 18 for SQL Server; cross-platform; supports Windows Auth, SQL Auth, and Azure AD
`sqlglot`	sqlglot	SQL parser	AST-level SELECT-only enforcement; rejects INSERT/UPDATE/DELETE/DDL at parse time
`pandas`	pandas	Data wrangling	DataFrames used for all intermediate computation (stats, pivots, merges)
`python-jose`	python-jose	JWT	HS256 token signing and verification; 24-hour token TTL
`bcrypt`	bcrypt	Password hashing	bcrypt with per-password salt; stored in users.json
`SlowAPI`	slowapi	Rate limiting	10 chat requests/minute/IP; wraps FastAPI with Redis-free in-memory limiter
`FastMCP`	mcp	MCP server	stdio-based Model Context Protocol server for Claude Code tool integration
`React 18`	react	Frontend framework	Component-based UI with hooks; Vite for build and HMR
`Tailwind CSS v4`	tailwindcss	Styling	Utility-first CSS; custom design tokens (carbon, snow, fog, pewter)
`Recharts`	recharts	Charts	Responsive SVG charts: bar, line, donut, area
`Axios`	axios	HTTP client	Auto-attaches JWT; 401 → redirect to login
`React Router v6`	react-router-dom	Client routing	SPA navigation; ProtectedRoute component enforces auth

5. Data Backend — SQLAttendanceStore

SQLAttendanceStore is the single source of truth for all attendance data. It is a read-only wrapper around a SQL Server database that provides four public methods used by both the API routes and the agent tools. It lives in src/sql_store.py.

5.1 Database Schema

The store expects a primary table named attendance in the configured primary database. Additional databases can be queried via the database parameter of run_sql_query.

```
CREATE TABLE attendance (
    student_id   INT           NOT NULL,
    student_name NVARCHAR(200),
    class        NVARCHAR(50),
    grade        NVARCHAR(20),
    date         DATE          NOT NULL,
    status       VARCHAR(10)   NOT NULL   -- 'present' | 'absent' | 'late' | 'excused'
);
```

The schema is flexible — the agent will adapt its T-SQL to whatever schema it discovers at runtime via run_sql_query. The structured tools (query_attendance, get_at_risk_students) assume the column names above.

5.2 SQL Safety

Every query executed by the store passes through _assert_select_only(), which uses sqlglot to parse the SQL into an abstract syntax tree (AST) and reject any statement that is not a single SELECT. The following statement types are explicitly blocked:

- INSERT — cannot add records
- UPDATE — cannot modify records
- DELETE — cannot remove records
- DROP — cannot remove tables or databases
- CREATE — cannot create objects
- ALTER — cannot change schema
- TRUNCATE — cannot clear tables
- MERGE — blocked to prevent upsert-style attacks
- Command — raw EXEC/xp_cmdshell etc. blocked
- Multiple statements — only a single SELECT per call

Additionally, every database accessed must be explicitly listed in the SQL_DATABASES environment variable. Queries against unlisted databases raise a PermissionError.

SQL Injection Prevention

- All user-supplied values (class names, date bounds, thresholds) are passed as parameterized query parameters via pyodbc — never string-interpolated into SQL.
- Only the group-by column expression is built from a hardcoded allow-list dict (_GROUP_EXPR) — never from user input.
- The sqlglot AST check catches multi-statement attacks (SELECT 1; DROP TABLE …) even if the parameterization were somehow bypassed.

5.3 TTL Cache

SQLAttendanceStore includes a simple in-process TTL cache (_TTLCache) with a 5-minute expiry. Both compute_stats() and get_at_risk() check the cache before hitting SQL Server. The cache key encodes all query parameters (group_by, period, classes, date bounds) so different filter combinations are cached independently. This reduces SQL Server load when multiple users view the same dashboard simultaneously.

5.4 Public Methods

Method	Description
`summary()`	Returns a dict with total_records, unique_students, date_range, overall_attendance_rate, total_absences, and classes list
`compute_stats(group_by, period, classes, date_from, date_to)`	Returns a DataFrame aggregated by the requested dimension and period; used by query_attendance, compare_periods, compare_classes, and the /data/stats endpoint
`get_at_risk(threshold, classes, date_from, date_to)`	Returns a DataFrame of students below the threshold, ordered by attendance rate ascending
`student_weekly_rates(student_ids, weeks)`	Returns a dict mapping student_id → list of weekly attendance rates (last N weeks); used for sparkline charts

5.5 Connection Management

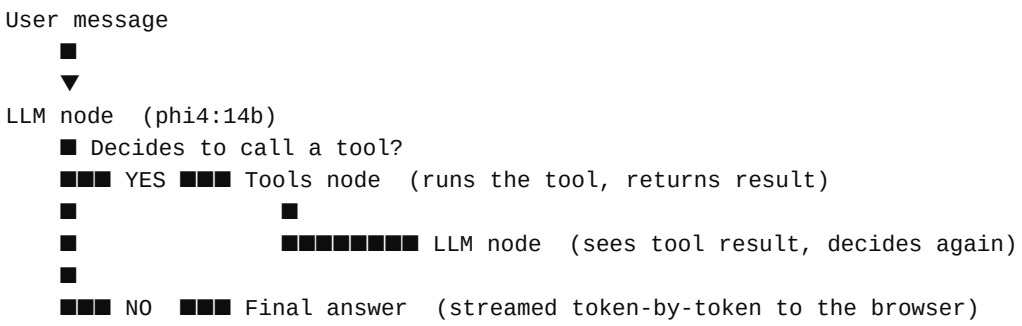
Each call to _query() opens a new pyodbc connection and closes it in a finally block. This avoids connection pool state issues across async workers but means one SQL Server round-trip per query. For high-traffic deployments, a connection pool (e.g. SQLAlchemy with pool_size) should be considered, but this is not needed for typical school-size loads.

6. AI Agent — ExcelsisAgent

ExcelsisAgent is the intelligence layer of Excelsis 360. It wraps a LangGraph ReAct (Reasoning + Acting) agent powered by phi4:14b running locally via Ollama. The agent decides which tools to call, in what order, based on the user's question and the results of previous tool calls.

6.1 The ReAct Loop

ReAct is a prompting pattern that interleaves Reasoning (thinking about what to do) and Acting (calling a tool or producing output). LangGraph implements this as a graph with two nodes — llm and tools — that alternate until the model decides to stop:



This loop repeats until the model emits a final AIMessage with no further tool calls. LangGraph manages the message list automatically, including injecting tool results as ToolMessage objects.

6.2 System Prompt

The agent receives a fixed system prompt that defines its persona, responsibilities, and tool usage rules. Key instructions include:

- Identity: "Expert School Attendance Analyst for Excelsis 360"
- Data integrity: never fabricate statistics; say so if data is unavailable
- Threshold: flag any class or student below 75% as at-risk
- Efficiency: call tools immediately — never narrate a tool call before making it
- SQL rules: T-SQL syntax only; SELECT-only; use TOP, DATEPART, FORMAT, CONVERT, ISNULL
- Dashboard rules: call update_dashboard_view when the user asks to see a chart or visual
- Conversation: answer greetings and general questions directly without calling tools

6.3 Conversation History

ExcelsisAgent maintains a rolling conversation history of up to 10 turns (20 messages: 10 human + 10 AI). The last 10 messages are prepended to every new request, giving the model context for follow-up questions. History can be cleared via the 'Clear history' button in the Chat UI, which calls reset_history().

6.4 Timeout Handling

Both the synchronous ask() method and the async astream_events() generator enforce a 90-second timeout. If the LLM does not complete within this window, the request fails gracefully with a user-facing message rather than hanging indefinitely. The synchronous path uses a daemon thread + threading.Event; the async path uses asyncio.timeout().

6.5 LLM Configuration

The ChatOllama instance is configured with:

Parameter	Value / Description
`model`	phi4:14b (default; overridden by MODEL env var)
`base_url`	http://localhost:11434 (Ollama's default port)
`temperature`	0.1 (low temperature for consistent, factual responses)
`num_ctx`	8192 tokens context window
`keep_alive`	10 minutes (model stays loaded in VRAM between requests)

7. Agent Tools

The agent has access to 7 tools defined in src/tools.py. Each tool is a Python function decorated with @tool from LangChain. The docstring is the tool description shown to the LLM. All tools receive a RunnableConfig from LangGraph that carries the SQLAttendanceStore instance.

query_attendance

Purpose: Answer a natural-language question about attendance statistics.

Parameters: query: str

Implementation: Parses the question for group_by (class, week, month, day_of_week, student_id, grade) and period (all, last_7_days, last_30_days) keywords, then calls store.compute_stats(). Returns a formatted DataFrame string.

Example calls:

```
"attendance by class"
"weekly trend last 30 days"
"which grade has lowest rate?"
```

get_at_risk_students

Purpose: Return students whose attendance rate is below a threshold.

Parameters: threshold: float = 75.0

Implementation: Calls store.get_at_risk(threshold). Returns a table of student_id, name, class, total, present, absent, late, attendance_rate ordered by rate ascending.

Example calls:

```
get_at_risk_students(threshold=70.0)
```

get_summary

Purpose: Return a high-level overview of all attendance data.

Parameters: (none)

Implementation: Calls store.summary() and returns the result as a JSON string. Includes total_records, unique_students, date_range, overall_attendance_rate, total_absences, classes.

Example calls:

```
get_summary()
```

update_dashboard_view

Purpose: Update the live dashboard to show a specific view or filter.

Parameters: classes: list[str], period: str, view: str

Implementation: Validates and returns a JSON payload. The SSE layer in ExcelsisAgent detects this tool's output and emits a dashboard_filter event to the browser, which updates the React dashboard state without any page reload.

Example calls:

```
update_dashboard_view(classes=["10A"], view="class")
```

```
update_dashboard_view(period="last_30_days", view="overview")
```

run_sql_query

Purpose: Execute an ad-hoc T-SQL SELECT query.

Parameters: sql: str, database: str = ""

Implementation: Passes the SQL through _assert_select_only() and the database allowlist check, then executes via store._query(). Automatically adds TOP 200 if no TOP clause is present. Returns up to 200 rows as a formatted table.

Example calls:

```
SELECT TOP 10 class, COUNT(*) AS absences FROM attendance WHERE status='absent' GROUP BY class ORDER
```

compare_periods

Purpose: Compare attendance rates between two time periods.

Parameters: period_a: str, period_b: str

Implementation: Calls compute_stats() twice (once per period), merges on class, and computes a delta column. Returns a side-by-side table.

Example calls:

```
compare_periods(period_a='last_7_days', period_b='last_30_days')
```

compare_classes

Purpose: Compare attendance statistics for two classes side by side.

Parameters: class_a: str, class_b: str

Implementation: Calls compute_stats() with classes=[class_a, class_b]. Returns a table showing total, present, absent, late, and rate for each class.

Example calls:

```
compare_classes(class_a='10A', class_b='10B')
```

8. REST API

The FastAPI backend is available at http://localhost:8000. Interactive OpenAPI docs are at http://localhost:8000/docs. All endpoints except /auth/login and /health require a valid Bearer JWT in the Authorization header.

8.1 Endpoints

Method	Path	Auth	Description
POST	/auth/login	None	Username + password → JWT access token (24h TTL)
GET	/auth/me	Any user	Returns current user's username
GET	/auth/users	Admin	List all registered usernames
POST	/auth/users	Admin	Create a new user account
DELETE	/auth/users/{username}	Admin	Delete a user (cannot delete 'admin')
POST	/chat/stream	Any user	SSE streaming chat — delegates to ExcelsisAgent.astream_events()
GET	/data/summary	Any user	Attendance data overview (calls store.summary())
GET	/data/at-risk	Any user	At-risk student list (threshold, classes, date_from, date_to params)
GET	/data/stats	Any user	Aggregated stats (group_by, period, classes, date_from, date_to params)
GET	/data/trends	Any user	Current and prior 30-day weekly stats for trend comparison chart
GET	/data/sparklines	Any user	Weekly rates for a list of student IDs (ids= CSV param)
GET	/health	None	Returns {"status": "ok"} — liveness probe

8.2 Rate Limiting

The POST /chat/stream endpoint is limited to 10 requests per minute per IP address using SlowAPI (a FastAPI-compatible port of Flask-Limiter). Exceeding the limit returns HTTP 429 with a Retry-After value. The rate limiter is in-process (no Redis required) — counters reset when the server restarts.

8.3 CORS

CORS is configured to allow requests from http://localhost:5173 (Vite dev server) and http://localhost:3000. In production, the React app is served as static files by FastAPI itself (via StaticFiles mount), eliminating the CORS requirement entirely.

8.4 Application Lifecycle

FastAPI's lifespan context manager handles startup and shutdown. On startup:

- ensure_default_admin() creates the admin account in users.json if it doesn't exist.
- SQLAttendanceStore() is instantiated and stored on app.state.store.
- ExcelsisAgent(store=store) is instantiated and stored on app.state.agent.
- _validate_startup() checks Ollama reachability and SQL Server connectivity, printing a status table.

The store and agent are shared across all requests via app.state — there is one instance per process. The agent's conversation history is shared across users in single-process deployments (this is intentional for a single-school use case).

9. Authentication & User Management

9.1 User Store

Users are stored in api/users.json as a flat dict mapping username → bcrypt hash. On first startup, ensure_default_admin() creates the admin account with the password from the ADMIN_PASSWORD env var (default: admin123). File writes use an atomic write-then-rename pattern (write to .tmp, then os.replace) to prevent corruption under concurrent writes.

```
{
  "admin": {
    "hashed_password": "$2b$12$..."
  },
  "ms_johnson": {
    "hashed_password": "$2b$12$..."
  }
}
```

9.2 JWT Tokens

Tokens are HS256 JWTs signed with JWT_SECRET. They contain two claims:

- sub — the username (used to reconstruct UserContext on each request)
- exp — expiry timestamp (24 hours from issuance)

decode_token() reconstructs a UserContext(user_id=username) from the token on every authenticated request. There is no token refresh mechanism — the user must re-login after 24 hours.

9.3 UserContext

UserContext is a minimal Python dataclass with a single field: user_id (the username string). It is threaded through all agent and tool calls via LangGraph's RunnableConfig so that tools can theoretically filter data by user. In the current implementation, there is no row-level filtering — all authenticated users see all data. The UserContext exists to make adding per-user filtering straightforward in the future.

9.4 Admin Operations

The 'admin' account is the only user that can access the Users management page and the /auth/users endpoints. The admin account cannot be deleted. New users can be created or deleted by the admin at any time. There is no password reset flow — an admin must delete and recreate the account.

10. React Web Interface

The frontend is a React 18 single-page application built with Vite and styled with Tailwind CSS v4. It follows a clean two-column layout (sidebar + main content) inspired by modern productivity tools.

10.1 Pages

Page	Access	Description
`Login (/login)`	Public	Username + password form. POSTs to /auth/login, stores JWT in localStorage, redirects to /dashboard.
`Chat (/chat)`	Authenticated	Natural-language chat interface. Streams responses token-by-token. Shows tool-use indicators. Displays contextual suggestion chips on first load.
`Dashboard (/dashboard)`	Authenticated	KPI cards + 5 interactive charts + at-risk student table + drilldown panel. Embeddable chat panel ('Ask Excelsis') can update dashboard filters live.
`Users (/users)`	Admin only	List, create, and delete user accounts. Admin cannot delete themselves.

10.2 Key Components

Component	Description
`Sidebar`	Left navigation with links to Chat, Dashboard, Users (admin only), and a logout button
`MessageBubble`	Renders a single chat turn. User messages align right; agent messages align left with markdown rendering. Shows tool-use badges (e.g. 'Querying attendance data…').
`ChatPanel`	Embedded chat panel on the Dashboard page. Same functionality as the Chat page but in a slide-in panel. Passes dashboard_filter events to the parent Dashboard.
`FilterBar`	Class multi-select and period dropdown. Changing either triggers a fresh data fetch for all dashboard charts.
`Breadcrumb`	Shows the current drill level (Overview → Class → Student). Click to navigate back up.
`DrilldownPanel`	Renders class-level or student-level detail depending on drill state. Student view shows a sparkline trend chart.
`AttendanceByClassChart`	Horizontal bar chart of attendance rate by class. Single-click filters; double-click drills down.
`WeeklyTrendChart`	Line chart of weekly attendance rate over the selected period.

Component	Description
`StatusDonutChart`	Donut chart of present / absent / late / excused proportions.
`WeekdayBarChart`	Bar chart of attendance rate by day of week.
`TrendComparisonChart`	Grouped bar chart comparing current vs prior 30-day periods per class.
`ProtectedRoute`	HOC that redirects unauthenticated users to /login.

10.3 Design System

The UI uses a minimal achromatic palette with a single accent colour. All design tokens are defined as Tailwind CSS custom properties:

Token	Hex	Usage
`carbon`	#0d0d0d	Primary text, headings, icons, filled buttons
`snow`	#ffffff	Page backgrounds, card surfaces
`fog`	#f9f9f9	Secondary backgrounds, sidebar, input fields, chart cards
`arctic-mist`	#ececec	Borders, dividers, hover backgrounds
`pewter`	#5d5d5d	Secondary text, placeholders, captions
`stone`	#8f8f8f	Tertiary text, inactive icons
`link-blue`	#007aff	Focus rings, interactive accents

10.4 SSE Client Implementation

The browser does not use the EventSource API (which doesn't support POST requests). Instead, streamChat() in web/src/api/client.ts uses the Fetch API with response.body as a ReadableStream. It reads chunks, splits on newlines, strips the 'data: ' prefix, and dispatches each event type to the appropriate callback.

10.5 Authentication Flow

- JWT is stored in localStorage under the key 'token'.
- The Axios instance (api/client.ts) automatically attaches it as 'Authorization: Bearer …' on every request.
- On 401 responses, the interceptor clears localStorage and redirects to /login.
- ProtectedRoute reads the token from localStorage — if absent, it redirects to /login immediately without hitting the server.

11. MCP Server

The Model Context Protocol (MCP) server exposes Excelsis 360's capabilities to AI-assisted development tools — primarily Claude Code. It runs as a stdio-based process, meaning it reads from stdin and writes to stdout using the MCP wire protocol. Identity is set at process startup via the MCP_USER_ID environment variable.

11.1 Exposed Tools

Tool Signature	Description
`ask_analyst(query)`	Ask the full ExcelsisAgent a natural-language question. The agent reasons across all 7 internal tools and returns a comprehensive answer. This is the most powerful MCP tool.
`attendance_summary()`	Returns a JSON summary (record count, students, date range, overall rate, classes). Fast — no LLM involved.
`at_risk_students(threshold)`	Lists students below the given attendance threshold as a formatted table. Default threshold: 75%.
`class_statistics(group_by, period)`	Returns attendance stats grouped by class, week, month, day_of_week, student_id, or grade, for the specified period.

11.2 Starting the MCP Server

```
# From the project root with .venv active:
MCP_USER_ID=ms_johnson python -m src.mcp_server

# Or with the default user (mcp_user):
python -m src.mcp_server
```

The server uses the same .env configuration as the web stack — SQL_SERVER, SQL_DATABASES, SQL_USERNAME, SQL_PASSWORD, and MODEL must all be set.

12. Jupyter Notebook

Excelsis.ipynb provides a fully interactive analysis environment. It shares the same src/ Python backend as the web application, so analysts get the same tools and agent but in a cell-by-cell exploration format.

12.1 Notebook Cells

Cell	Name	Description
1	Setup & imports	Load environment variables, import src modules
2	Connectivity check	Verify Ollama is running at http://localhost:11434; fail fast if not
3	Connect to SQL Server	Instantiate SQLAttendanceStore and print summary
4	Data summary	Call store.summary() and display as a formatted dict
5	Set analyst identity	Set CURRENT_USER = UserContext(user_id='...') — determines the identity used for all agent calls
6	Agent setup	Instantiate ExcelsisAgent(store=store) with the SQL backend
7	Chat interface	Call agent.ask(query, user=CURRENT_USER) for natural-language queries
8	Tool calls directly	Call individual tools (query_attendance, get_at_risk_students, etc.) bypassing the LLM
9	Visualisations	Generate Plotly charts and matplotlib dashboards from the store data

12.2 Changing the Analyst Identity

Edit CURRENT_USER in Cell 5 to simulate a different staff member:

```
CURRENT_USER = UserContext(user_id="ms_johnson")
# or
CURRENT_USER = UserContext(user_id="admin")
```

In the current implementation, all users see all data. Setting CURRENT_USER primarily affects the agent's conversation context and would enable per-user data filtering if such logic were added to the tools.

13. Configuration Reference

All configuration is via environment variables, typically set in .env (loaded by python-dotenv at startup). The .env.example file documents all variables.

Variable	Required	Default	Description
`SQL_SERVER`	Yes	—	SQL Server hostname, IP, or named instance (e.g. myserver\SQLEXPRESS)
`SQL_DATABASES`	Yes	—	Comma-separated list of databases the agent is allowed to query
`SQL_USERNAME`	If sql auth	—	SQL Server login username
`SQL_PASSWORD`	If sql auth	—	SQL Server login password
`SQL_PRIMARY_DB`	No	First in SQL_DATABASES	Default database for queries that don't specify one
`SQL_AUTH_METHOD`	No	sql	Authentication method: sql | windows | azure_ad
`SQL_DRIVER`	No	{ODBC Driver 18 for SQL Server}	ODBC driver string
`JWT_SECRET`	Yes (prod)	change-me-in-production	Secret key for JWT signing — MUST be changed in production
`ADMIN_PASSWORD`	No	admin123	Initial password for the admin account (only used on first startup)
`MODEL`	No	phi4:14b	Ollama model name for the ReAct agent
`AT_RISK_THRESHOLD`	No	75.0	Default attendance percentage threshold for at-risk flagging
`MCP_USER_ID`	No	mcp_user	Username used by the MCP server process

14. Security Model

14.1 What Is Enforced

• **SQL read-only enforcement** — sqlglot AST parsing rejects any non-SELECT statement at the store layer, before the query reaches SQL Server.

• **Database allowlist** — only databases listed in SQL_DATABASES can be queried; all others raise a PermissionError.

• **SQL injection prevention** — all user-supplied values are parameterized; group-by columns come from a hardcoded allow-list.

• **JWT authentication** — all API endpoints except /auth/login and /health require a valid, unexpired JWT.

• **Password hashing** — all passwords are bcrypt-hashed with per-password salts; plaintext passwords are never stored.

• **Rate limiting** — chat endpoint is limited to 10 requests/minute/IP to prevent abuse.

• **Local inference** — all LLM inference runs on localhost via Ollama; attendance data never leaves the network.

• **Timeout enforcement** — 90-second hard timeout on all agent calls prevents hanging requests.

14.2 What Is Not Enforced

Known Limitations

• No row-level security — all authenticated users see all student data. There is no per-teacher or per-class access control.

• Shared agent history — a single ExcelsisAgent instance serves all users; conversation history is shared across sessions in single-process deployments.

• users.json is not encrypted — the file contains bcrypt hashes which are computationally expensive to crack, but the file should be protected by filesystem permissions.

• No HTTPS — the server runs plain HTTP by default. In production, put Nginx or Caddy in front with TLS termination.

• No audit log — there is no record of which user asked which questions or ran which SQL queries.

• JWT_SECRET default — the default 'change-me-in-production' secret must be changed before any production deployment.

• No CSRF protection — the API is stateless (JWT, no cookies) so traditional CSRF does not apply, but the CORS allow-list should be tightened in production.

15. Running the Stack

15.1 Prerequisites

- Python 3.11 or later
- Node.js 18 or later + npm
- Ollama installed and running (https://ollama.com)
- SQL Server accessible from the host machine
- ODBC Driver 18 for SQL Server installed on the host

15.2 First-Time Setup

```
# 1. Pull the LLM (one-time, ~8GB download)
ollama pull phi4:14b

# 2. Copy and configure environment
cp .env.example .env
# Edit .env: set SQL_SERVER, SQL_DATABASES, SQL_USERNAME, SQL_PASSWORD, JWT_SECRET

# 3. Python virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.lock   # use lock file for reproducible installs

# 4. Frontend dependencies
cd web && npm install && cd ..
```

15.3 Starting Both Servers

```
bash start.sh
# FastAPI  → http://localhost:8000
# React    → http://localhost:5173
# Press Ctrl-C to stop both
```

15.4 Starting Servers Individually

```
# Backend only
source .venv/bin/activate
uvicorn api.main:app --reload

# Frontend only (separate terminal)
cd web && npm run dev
```

15.5 Default Login

- URL: http://localhost:5173

- Username: admin
- Password: value of ADMIN_PASSWORD in .env (default: admin123)

15.6 Startup Validation Output

On startup, the backend prints a validation summary:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Excelsis 360 — startup
  Model:      phi4:14b                OK
  SQL Server: myserver                OK
  Databases:  attendance_db,reports_db
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

UNREACHABLE indicates either Ollama is not running (check: ollama list) or SQL Server credentials are incorrect.

16. Test Suite

Tests live in tests/test_qa.py. The suite is split into two classes with distinct requirements:

16.1 Unit Tests — TestZeroKnowledge

These tests call LangGraph tools directly with a synthetic AttendanceDataStore seeded from a pandas DataFrame. No Ollama, no SQL Server, no network required. They run in under a second and verify:

- query_attendance returns data containing expected class names
- get_summary returns JSON with required keys (total_records, overall_attendance_rate)
- Tools return a graceful message when no store is connected
- get_at_risk_students correctly identifies students below the threshold
- update_dashboard_view returns valid JSON with the expected payload shape

```
pytest tests/test_qa.py -v
```

16.2 Integration Tests — TestModelStress

These tests require a live Ollama instance with phi4:14b loaded. They verify:

- A multi-part complex query completes within 120 seconds and returns >50 characters
- A greeting does not trigger tool calls and returns a conversational response

```
pytest tests/test_qa.py -v -m integration   # integration tests only
pytest tests/test_qa.py -v --run-all         # everything
```

17. Extending the System

17.1 Adding a New Database

- Add the database name to SQL_DATABASES in .env (comma-separated).
- The agent can immediately query it via run_sql_query(sql=..., database='new_db').
- Update the system prompt in src/agent.py to tell the model what tables are available in the new database.

17.2 Adding a New Agent Tool

1. Define a function in src/tools.py and decorate it with @tool:

```
@tool
def my_new_tool(param: str, config: RunnableConfig = None) -> str:
    """Describe what the tool does — this is shown to the LLM."""
    store = _store(config)
    ...
    return result_string
```

2. Add it to ALL_TOOLS in src/tools.py.

3. Add a description to the Tool usage section of SYSTEM_PROMPT in src/agent.py.

17.3 Changing the LLM

Any Ollama-hosted model that supports tool calling can be used. Set MODEL in .env. The model must support function/tool calling — check the Ollama model page for 'tools' support. Recommended alternatives: llama3.1:8b (faster, less accurate), mistral-small (smaller context), qwen2.5:14b.

17.4 Adding a New Frontend Page

- Create web/src/pages/MyPage.tsx following the pattern of existing pages.
- Add a route in web/src/App.tsx: } />
- Add a link in web/src/components/Sidebar.tsx.
- If the page needs data, add a new endpoint in api/routers/ and include_router() in api/main.py.

17.5 Adding Per-User Data Filtering

UserContext is already threaded through every tool call via RunnableConfig. To add per-user filtering:

- In SQLAttendanceStore, add a user_id column to the attendance table and a user→classes mapping.
- In each tool, extract user_id from config['configurable']['user_context'].user_id.
- Pass allowed classes as an additional filter to store._query().

- No changes to the agent or API layer are needed.

17.6 Production Deployment

- Set JWT_SECRET to a long random string (e.g. openssl rand -hex 32).
- Set ADMIN_PASSWORD to a strong password.
- Build the React app: cd web && npm run build
- Mount the built files via FastAPI StaticFiles — remove the CORS middleware.
- Run uvicorn with --workers 4 (multiple workers share the same SQL store but have separate agent instances and conversation histories).
- Put Nginx or Caddy in front for TLS termination.
- Restrict users.json file permissions: chmod 600 api/users.json